

INFRASTRUCTURE AS CODE

AINDA ESTAMOS TE LEVANDO ÀS NUVENS



Caroline Perez Gonçalves
2026

INFRASTRUCTURE AS CODE

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	4
INTRODUÇÃO	5
O QUE É INFRASTRUCTURE AS CODE	6
INTRODUÇÃO AO TERRAFORM.....	7
INTRODUÇÃO AO PULUMI	9
TERRAFORM X PULUMI	10
PROJETO PRÁTICO	11
AGRADECIMENTO.....	15
CONTATO.....	16

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

Nesta apostila, vamos dar mais um passo na nossa jornada pela Cloud Computing e conhecer o **Infrastructure as Code, ou IaC**. Depois de entender os fundamentos da nuvem e conhecer plataformas como AWS e Azure, chegou a hora de entender como podemos automatizar a criação e o gerenciamento da infraestrutura utilizando código.

Ao longo dos estudos, vamos entender o que é Infrastructure as Code, por que essa abordagem se tornou tão importante e conhecer duas ferramentas bastante utilizadas nesse universo: **Terraform e Pulumi**. A ideia não é sair decorando comandos ou transformar esta apostila em um curso completo de cada ferramenta. Primeiro vamos entender os conceitos e depois conhecer como cada uma trabalha.

Quero que você aproveite esta apostila sem pressa. Se algum conceito ou ferramenta parecer complicado no começo, **acalme-se, tá tudo bem**. Vamos entender cada coisa aos poucos e relacionar o conteúdo com situações que fazem sentido no mundo real. Afinal, aprender tecnologia fica muito mais fácil quando conseguimos entender o problema antes de pensar na ferramenta.

Então, prepare-se para dar mais um passo na nossa jornada e descobrir como o código pode deixar a criação e o gerenciamento de infraestrutura muito mais organizados e automatizados.

Sente-se, relaxe e sinta-se em casa. E não se preocupe, desta vez nós não vamos precisar subir às nuvens.

Boa sorte e muito sucesso!

INTRODUÇÃO

Afinal, por que aprender sobre Infrastructure as Code?

Caso você tenha lido até o fim a apresentação dessa apostila, percebeu que nós não vamos precisar subir até às nuvens. E esse comentário vai fazer muito sentido no fim dessa apostila. A resposta para essa e outras perguntas tem um segredo. Muita gente ouve falar em Infrastructure as Code e já imagina um assunto complicado, cheio de códigos, ferramentas e configurações que parecem existir apenas para quem trabalha há anos com Cloud. Mas isso é uma mentira.

Talvez você já esteja esperando a pergunta que eu sempre faço nas minhas introduções.

Como assim, Carol?

Quando falamos em Infrastructure as Code, estamos falando de uma forma de criar e gerenciar infraestrutura utilizando código. E não são códigos absurdos! Em vez de fazer tudo manualmente, podemos definir o que precisamos e deixar que uma ferramenta faça esse trabalho de forma automatizada.

E isso pode fazer uma diferença enorme na otimização do tempo e na segurança daquilo que está sendo criado.

Imagine precisar criar a mesma estrutura várias vezes. Fazer tudo manualmente pode levar bastante tempo e também aumentar as chances de esquecer alguma configuração. Com IaC, podemos transformar essa infraestrutura em código e reutilizá-la sempre que necessário. Isso é vida!

Quando você entende isso, percebe que aprender IaC não significa simplesmente decorar comandos de uma ferramenta. Significa entender como automatizar a infraestrutura, tornar processos mais organizados, repetíveis e facilitar o gerenciamento dos ambientes.

Ao longo desta apostila, vamos conhecer os principais conceitos de Infrastructure as Code e trabalhar com duas ferramentas bastante conhecidas nesse universo: Terraform e Pulumi. Vamos entender como elas funcionam, o que possuem em comum, quais são suas diferenças e onde cada uma pode fazer sentido.

Claro, existem muitos assuntos mais avançados dentro de IaC que exigem bastante estudo e prática. Mas, como sempre, vamos começar pelo começo. Primeiro vamos entender o conceito. Depois, vamos conhecer as ferramentas e, no final, colocar tudo em prática.

Acalme-se, tá tudo bem. A infraestrutura não vai se criar sozinha... ainda.

Vamos lá.

O QUE É INFRASTRUCTURE AS CODE

Até agora, quando falamos de Cloud, vimos que podemos criar máquinas virtuais, bancos de dados, redes, armazenamento e vários outros recursos. Mas imagine que você precise criar toda essa estrutura novamente. E depois novamente. E depois mais uma vez. E mais uma.

Fazer tudo manualmente pode funcionar quando estamos trabalhando com poucos recursos, mas conforme a infraestrutura cresce, isso começa a ficar trabalhoso e, principalmente, aumenta a possibilidade de erros (eu mesma iria errar).

É aí que entra o **Infrastructure as Code**, ou simplesmente **IaC**.

A ideia é bem direta: em vez de criar e configurar a infraestrutura manualmente, nós descrevemos essa infraestrutura utilizando código.

Pense em uma situação simples. Você precisa criar uma máquina virtual com determinada quantidade de memória, processamento, sistema operacional, rede e algumas configurações específicas. Em vez de entrar no portal da sua Cloud escolhida, clicar em várias opções e repetir todo esse processo sempre que precisar de uma nova máquina, podemos escrever essas configurações em um arquivo e usar quando precisar.

A ferramenta utilizada para IaC lê esse código e cria a infraestrutura de acordo com aquilo que foi definido.

Isso traz uma vantagem enorme, pois podemos repetir o processo de maneira muito mais rápida e consistente.

Imagine que uma empresa precise criar três ambientes diferentes: desenvolvimento, homologação e produção.

Fazer tudo manualmente em cada ambiente pode gerar diferenças entre eles. Alguém pode esquecer uma configuração, escolher uma opção diferente ou simplesmente clicar no lugar errado.

Com IaC, podemos definir a infraestrutura como código e utilizar essa definição para criar ambientes seguindo o mesmo padrão. Tudo igualzinho.

Infrastructure as Code também ajuda na **padronização, automação, repetibilidade e controle das mudanças** realizadas na infraestrutura.

E aqui aparece uma conexão importante com aquilo que já vimos.

Se o código que descreve a infraestrutura pode ser armazenado em um sistema de versionamento, como o Git, podemos acompanhar alterações, saber quem modificou determinada configuração e até recuperar versões anteriores. Ou seja, a infraestrutura começa a ser tratada de uma forma muito parecida com o desenvolvimento de software.

Não é só apertar um botão e o milagre acontece. Muita gente tem isso em mente, mas precisa urgentemente parar com isso. Nós ainda precisamos saber o que estamos

fazendo, entender a arquitetura, definir corretamente os recursos e revisar as configurações. A ferramenta automatiza o processo, mas não substitui o conhecimento de quem está criando a infraestrutura.

Outro ponto importante é que IaC não é uma ferramenta. **Infrastructure as Code é uma abordagem.**

Assim como em outros assuntos existem abordagens e métodos, aqui também.

Terraform e Pulumi, que vamos conhecer nesta apostila, são ferramentas que permitem aplicar essa abordagem.

Existem outras ferramentas e tecnologias no mercado, mas nosso foco aqui será entender os conceitos e conhecer essas duas soluções.

No final, o mais importante é você olhar para uma infraestrutura e começar a pensar:

"Eu realmente preciso criar tudo isso manualmente?"

Se a resposta for não, talvez seja hora de colocar essa infraestrutura em código e ser feliz.

INTRODUÇÃO AO TERRAFORM

Agora que já entendemos o que é Infrastructure as Code, chegou a hora de conhecer uma das ferramentas mais conhecidas desse universo: o **Terraform**.

Eu sinto que ele é o meu amorzinho. Então, ignorem caso eu fale mais dele do que de outra ferramenta. Pois, caso você tenha visto outras apostilas, eu cheguei a comentar que muitos assuntos eu estudei para o meu primeiro trabalho na faculdade. E a HashiCorp eu vi naquela época também.

O Terraform é uma ferramenta de Infrastructure as Code originalmente criada pela HashiCorp. Atualmente, a HashiCorp faz parte da IBM há uns 2 anos talvez (estou escrevendo isso em 2026).

Sua principal função é permitir que você **defina sua infraestrutura utilizando código** e, a partir dessa definição, possa criar e gerenciar recursos em diferentes ambientes de Cloud.

A ideia é simples. Em vez de entrar no portal da AWS, Azure ou outro provedor e criar cada recurso manualmente, você descreve no código aquilo que deseja ter.

Por exemplo, imagine que você precise de uma máquina virtual, uma rede e um armazenamento. Em vez de configurar cada recurso manualmente, você pode escrever essas informações em arquivos do Terraform.

Depois, o Terraform interpreta essa configuração e trabalha para deixar a infraestrutura de acordo com aquilo que foi definido.

E aqui aparece um detalhe muito legal e importante: o Terraform trabalha de forma

declarativa.

Isso significa que você descreve o que quer e ele vai lá e faz. Você não precisa dizer “Terraform, primeiro faz isso e depois aquilo”. Você simplesmente diz o que precisa e ele cria! Claro, escrevendo em padrões que ele entenda.

Parece uma diferença pequena, mas ela é muito importante quando começamos a trabalhar com infraestrutura maior.

Outro ponto interessante é que o Terraform não é limitado a um único provedor de Cloud. Ele pode trabalhar com diferentes plataformas e serviços por meio dos chamados **providers**.

Isso significa que podemos utilizar Terraform para trabalhar com recursos da AWS, Azure, Google Cloud e diversos outros serviços. Eles são os providers!

E aqui está uma das coisas mais interessantes sobre aprender Terraform: você começa a perceber que a ferramenta fica entre você e os diferentes provedores.

Os serviços continuam sendo diferentes, mas a ideia de definir a infraestrutura utilizando código permanece.

O Terraform também permite visualizar quais mudanças serão realizadas antes de aplicá-las. Isso é muito importante porque, antes de sair criando ou alterando recursos, conseguimos analisar o que a configuração pretende modificar.

Analisar antes de subir o que fizemos é muitooooo importante. Parece óbvio, mas em tecnologia é melhor conferir duas vezes antes de descobrir que aquele botão realmente fazia o que você achava.

Outro conceito importante é o **state**, ou estado do Terraform. De forma simplificada, ele ajuda o Terraform a acompanhar os recursos que ele está gerenciando e relacioná-los com aquilo que foi definido no código.

Não precisamos e nem vamos entrar profundamente nesse assunto agora. O importante neste momento é entender que o Terraform precisa saber qual é a situação atual da infraestrutura para conseguir comparar com aquilo que você declarou.

E é justamente essa combinação que torna o Terraform tão útil: **código, automação, controle e repetibilidade**.

Nesta apostila, não vamos transformar o assunto em um curso completo de Terraform. A ideia é entender seus principais conceitos e compreender como ele se encaixa dentro do Infrastructure as Code, consequentemente, em Cloud.

E a documentação do Terraform é incrível! Completa, simples e direta.

Bom, depois disso, vamos conhecer outra abordagem com o **Pulumi** e entender como as duas ferramentas se comparam.

Porque sim, existem outras formas de fazer IaC. Talvez não seja tão empolgante quanto o Terraform, mas é justamente aí que a coisa começa a ficar interessante.

Bora!

INTRODUÇÃO AO PULUMI

Agora que já conhecemos o Terraform, vamos conhecer outra ferramenta bastante interessante para trabalhar com **Infrastructure as Code: o Pulumi**.

A ideia continua sendo a mesma que vimos até aqui: utilizar código para criar e gerenciar infraestrutura, evitando depender exclusivamente de configurações feitas manualmente nos providers.

Mas existe uma diferença que chama bastante atenção.

Enquanto o Terraform utiliza a própria linguagem de configuração, o **Pulumi permite definir infraestrutura utilizando linguagens de programação tradicionais**, como Python, TypeScript, JavaScript, Go, C# e Java (deve ser por isso que eu gosto mais do Terraform).

E isso muda bastante a experiência de quem está escrevendo a infraestrutura.

Imagine que você já trabalha com Python. Em vez de precisar aprender uma linguagem específica para começar a definir seus recursos, você pode utilizar conhecimentos que já possui na linguagem para trabalhar com infraestrutura.

Isso pode ser especialmente interessante para equipes que já possuem uma forte cultura de desenvolvimento e querem aproximar o código da aplicação do código responsável pela infraestrutura.

Mas calma. Isso também não significa que o Pulumi seja simplesmente "Terraform usando Python". Longe disso, por favor.

As ferramentas possuem propostas e características próprias, e é justamente isso que vamos entender melhor.

No Pulumi, podemos definir recursos, configurações e relacionamentos entre eles utilizando código. A ferramenta então transforma essas definições em ações realizadas nos provedores de infraestrutura escolhidos.

Assim como acontece com o Terraform, o Pulumi também pode trabalhar com diferentes provedores de Cloud. Ou seja, novamente encontramos aquela ideia que já estamos construindo desde o início desta trilha: os provedores mudam, os serviços mudam, mas os conceitos de infraestrutura continuam presentes .

Outra característica interessante é que, por utilizar linguagens de programação, o Pulumi permite trabalhar com recursos que desenvolvedores já conhecem, como variáveis, funções, estruturas condicionais e outros recursos da linguagem escolhida.

Isso pode deixar algumas soluções mais flexíveis (não sei se a palavra flexível atende bem o que eu quero dizer). Mas também existe um ponto importante: utilizar uma linguagem de programação completa não significa automaticamente que tudo ficará mais simples.

Para quem não possui familiaridade com programação, pode acontecer justamente o contrário. Por isso, a escolha da ferramenta também depende do perfil da equipe, dos conhecimentos disponíveis e das necessidades do projeto.

E é aqui que começamos a chegar em uma pergunta interessante:

Se Terraform e Pulumi conseguem trabalhar com Infrastructure as Code, qual deles devemos usar?

A resposta não é simplesmente "um é melhor que o outro". Eu queria que fosse, mas preciso aceitar que não é.

Cada ferramenta possui características diferentes, e a melhor escolha depende do cenário. É justamente isso que vamos analisar agora.

TERRAFORM X PULUMI

Agora que já conhecemos as duas ferramentas, chegou a hora de colocar as duas lado a lado. E antes que você pense que eu vou escolher uma vencedora, já aviso: não existe uma ferramenta que seja melhor para absolutamente todos os cenários.

Terraform e Pulumi têm o mesmo grande objetivo: trabalhar com **Infrastructure as Code**, permitindo criar e gerenciar infraestrutura por meio de código. O que muda é principalmente a forma como fazemos isso.

O Terraform utiliza a HashiCorp Configuration Language (HCL) para definir os recursos. É uma linguagem criada para trabalhar com configurações e infraestrutura e, justamente por isso, possui uma estrutura bastante voltada para esse objetivo. A escrita é declarativa! Não esqueçam disso: você diz o que precisa e ela faz.

O Pulumi segue uma proposta diferente. Ele permite utilizar linguagens de programação que já conhecemos, como Python, JavaScript, Go, C#, entre outras.

Na prática, isso significa que uma pessoa que já trabalha com Python, por exemplo, pode utilizar seus conhecimentos da linguagem para definir sua infraestrutura utilizando Pulumi.

E aqui temos uma diferença importante.

No Terraform, você aprende uma linguagem própria para trabalhar com a configuração da infraestrutura. No Pulumi, você pode utilizar uma linguagem de programação que já conhece.

Isso pode ser uma vantagem para equipes formadas principalmente por desenvolvedores. Por outro lado, o Terraform possui uma abordagem muito consolidada e bastante conhecida no mercado, o que também pode pesar na escolha.

Outro ponto importante é a comunidade e o ecossistema que existe.

O Terraform possui uma grande adoção no mercado e uma enorme quantidade de providers e módulos disponíveis. Isso facilita encontrar exemplos, documentação, soluções para problemas comuns e profissionais que já conhecem a ferramenta.

O Pulumi também possui um ecossistema bastante amplo e permite trabalhar com diferentes provedores, mas sua proposta é diferente e pode se encaixar melhor em determinados ambientes.

Então, como escolher?

Se sua equipe já trabalha bastante com Terraform, possui experiência com HCL e tem uma estrutura baseada nessa ferramenta, provavelmente não existe motivo para trocar simplesmente porque outra ferramenta apareceu.

Da mesma forma, se sua equipe possui forte conhecimento em linguagens como Python ou TypeScript e quer utilizar esse conhecimento também na infraestrutura, o Pulumi pode ser uma alternativa interessante.

Também precisamos considerar manutenção, conhecimento da equipe, documentação, a integração com o ambiente existente e as necessidades do projeto. Ou seja, não escolha uma ferramenta simplesmente porque alguém disse que ela é "a melhor".

Não esqueça: entenda o cenário primeiro.

É como escolher uma ferramenta na caixa de ferramentas de casa. Você não pega uma furadeira para apertar um parafuso só porque ela é mais tecnológica. Às vezes, a chave de fenda resolve o problema muito melhor.

Outro ponto importante é que aprender uma ferramenta não significa ficar preso a ela para sempre. **As coisas mudam!**

O mais importante é compreender os fundamentos de **Infrastructure as Code**. Quando você entende conceitos como infraestrutura declarativa, automação, versionamento, estado, providers e gerenciamento de recursos, fica muito mais fácil aprender outras ferramentas depois.

PROJETO PRÁTICO

ESTUDE COMIGO COM IaC

Até aqui, aprendemos o que é Infrastructure as Code, conhecemos o Terraform e também vimos o Pulumi. Agora vamos visualizar como esses conceitos poderiam ser utilizados em um cenário real.

Vamos imaginar novamente a nossa querida **Estude Comigo**.

A plataforma precisa de alguns recursos na AWS. Normalmente, poderíamos acessar o console da AWS, procurar o serviço desejado, preencher as configurações e criar cada recurso manualmente. Funciona? Claro que sim.

Mas e se precisássemos repetir essa estrutura várias vezes?

É justamente aí que o IaC começa a fazer sentido.

Vamos ver isso agora.

Criando uma EC2

Imagine que precisamos criar uma máquina virtual EC2 para executar uma aplicação da Estude Comigo.

De forma manual, entraríamos no console da AWS, acessaríamos o serviço EC2, escolheríamos a imagem, o tipo da instância, configurações de rede, armazenamento e outras opções. Depois, revisariamos tudo e criaríamos a máquina.

Com o Terraform, poderíamos descrever essa infraestrutura em código. Como seria? Um exemplo simplificado seria:

```
resource "aws_instance" "estude_comigo" {
  ami = "ami-xxxxxxx"
  instance_type = "t3.micro"
  tags = {
    Name = "estude-comigo"
  }
}
```

Atenção: Esse exemplo não representa uma configuração completa de produção. A ideia aqui é apenas visualizar como podemos declarar que queremos uma instância EC2. E você não precisa decorar isso, porque a documentação tem todos os modelos para você seguir. Claro, com o tempo você acostuma e para de consultar sempre.

Agora vamos olhar para a mesma ideia utilizando Pulumi com Python:

```
import pulumi
import pulumi_aws as aws

instance = aws.ec2.Instance(
    "estude-comigo",
    instance_type="t3.micro",
    ami="ami-xxxxxxx"
)
```

Perceba a diferença.

No Terraform, estamos utilizando a HCL para declarar o recurso. No Pulumi, estamos utilizando uma linguagem de programação, neste exemplo, Python.

O objetivo continua sendo o mesmo: **descrever a infraestrutura que queremos criar**.

E, assim como no exemplo do Terraform, a AMI apresentada é apenas ilustrativa para essa explicação e precisaria ser substituída por uma AMI válida para a região escolhida.

Criando um armazenamento

Agora imagine que a Estude Comigo precisa de um local para armazenar suas apostilas e outros arquivos. Na AWS, poderíamos utilizar um bucket do **Amazon S3**.

Com Terraform:

```
resource "aws_s3_bucket" "apostilas" {  
  bucket = "estude-comigo-apostilas"  
}
```

Com Pulumi:

```
bucket = aws.s3.Bucket(  
    "estude-comigo-apostilas"  
)
```

Novamente, a ideia é parecida.

Estamos declarando que queremos um recurso de armazenamento, mas utilizando duas ferramentas diferentes.

Atenção: o nome apresentado é apenas um exemplo. Na prática, o nome do bucket precisa seguir as regras de nomenclatura do S3 e ser único globalmente.

Criando um banco de dados

Agora imagine que a plataforma precise de um banco de dados para armazenar informações dos alunos, assinaturas e outros dados da aplicação. Na AWS, poderíamos utilizar o **Amazon RDS**.

No Terraform, teríamos uma configuração semelhante a:

```
resource "aws_db_instance" "estude_comigo" {  
  engine      = "mysql"  
  instance_class = "db.t3.micro"  
  allocated_storage = 20  
}
```

No Pulumi, utilizando Python:

```
database = aws.rds.Instance(  
    "estude-comigo",  
    engine="mysql",  
    instance_class="db.t3.micro",  
    allocated_storage=20
```

)

Novamente, estamos apenas **visualizando a ideia**. Esses exemplos foram simplificados para que possamos enxergar o conceito sem entrar em configurações que ainda não estudamos.

Em uma configuração real, precisamos configurar o provider correspondente para que o Terraform saiba com qual plataforma ou serviço deverá trabalhar. A autenticação e as credenciais também precisam ser configuradas de forma adequada. Não é só chegar e fazer. Existem vários detalhes, mas isso tudo é o começo e não vamos dificultar nada.

E essa é justamente a ideia deste projeto: ser simples.

Não quero que você saia daqui copiando esses códigos e tentando criar recursos na AWS.

Neste momento, o objetivo é apenas perceber a diferença entre uma configuração manual e a abordagem de Infrastructure as Code.

Quando fazemos tudo pelo console, precisamos repetir os passos manualmente.

Quando utilizamos IaC, podemos transformar essa infraestrutura em código, versioná-la, reutilizá-la e modificar sua configuração de maneira mais organizada.

É essa mudança de pensamento que eu quero que você leve desta apostila.

Não estamos apenas criando uma EC2, um bucket ou um banco de dados. Estamos aprendendo a descrever infraestrutura como código para aproveitar seus benefícios.

E, futuramente, quando entrarmos em projetos mais completos, esses conceitos vão começar a fazer ainda mais sentido.

Prepare-se.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.